

C# and Object-Oriented Programming Interview & Concept Guide

1. Introduction - OOPS and Basics

Introduction

- **Duration:** 0:40

- **Q1. What is C#? What is the difference between C# and .NET?**

Discover the difference between C# and .NET, where C# is an object-oriented language running on the .NET framework, with CLR, framework class libraries, and assemblies.

- **Q2. What is OOPS? What are the main concepts of OOPS?**

Explore the core concepts of object-oriented programming, including objects, classes, inheritance, polymorphism, abstraction, and encapsulation, and understand how OOP provides a clear structure for software and web applications.

- **Q3. What are the advantages of OOPS?**

- **Q4. What are the limitations of OOPS?**

Identify the limitations of OOP concepts in small applications, highlighting the need for planning and time when implementing inheritance, and balancing OOP ideals with execution and delivery.

- **Q5. What are Classes and Objects?**

Duration: 3:46

Learn how classes and objects work in C#, where a class acts as a blueprint with fields, properties, methods, and a constructor, and objects are the in-memory instances.

- **Q6. What are the types of classes in C#?**

Duration: 0:24

Discover the types of classes in C#, including public, private, protected, abstract,

static, partial, and sealed, and learn how to answer this interview question from both perspectives.

- **Q7. Is it possible to prevent object creation of a class in C#?**

- **Q8. What is Property?**

Define a property as a class member that provides a flexible mechanism to read and write private fields, and enforce encapsulation by avoiding direct field access.

- **Q9. What is the difference between Property and Function?**

- **Q10. What are Namespaces?**

Duration: 1:00

Understand how namespaces act as containers for related classes and other types, and learn to import and access classes such as StringBuilder from System.Text.

2. OOPS & C# - Inheritance, Abstraction, Encapsulation & Polymorphism

Introduction

Explore the four pillars of OOP: inheritance, abstraction, encapsulation, and polymorphism through 16 scenario-based questions. Experience about 30 minutes of content with adjustable playback speed.

- **Q11. What is Inheritance? When to use Inheritance?**

Learn how inheritance creates a parent-child relationship that lets a derived class automatically reuse base class methods, improving code reusability and avoiding duplication.

- **Q12. What are the different types of Inheritance?**

Explore the different types of inheritance in C#, including single, multiple (via interfaces), multi-level, and hierarchical inheritance, with base and derived classes.

- **Q13. Does C# support Multiple Inheritance? How do you implement it?**

- **Q14. How to prevent a class from being Inherited?**

Explain how to prevent inheritance in C# by using sealed on a base class, contrast with using static for non-inheritable classes, and note that sealed allows objects while static cannot.

- **Q15. Are private class members inherited to the derived class?**

Explain that private class members are not inherited by derived classes. Note that only public and protected members can be inherited.

- **Q16. What is Abstraction? How to implement abstraction in real applications?**

- **Q17. What is Encapsulation? How to implement encapsulation in real applications?**

- **Q18. What is the difference between Abstraction and Encapsulation?**

- **Q19. What is Polymorphism and what are its types? When to use polymorphism?**

Duration: 2:28

Explains polymorphism and its types, including method overloading (compile-time) and overriding (runtime), with examples in C# and practical guidelines on when to use polymorphism in real applications.

- **Q20. What is Method Overloading? In how many ways can a method be overloaded?**

Explore method overloading in C# as a form of polymorphism, where same-name methods perform different tasks. Learn three overloads: different parameter counts, types, and orders.

- **Q21. When should you use method overloading in real applications?**

Learn when to use method overloading in real projects to cover scenarios without proliferating unique method names, using examples like Add and string substring overloads.

- **Q22. If two methods have the same signature except return type, are the methods overloaded?**

- **Q23. What is the difference between Overloading and Overriding?**

Explore the difference between method overloading and method overriding using base and derived classes, virtual and override keywords, and runtime versus compile-time polymorphism.

- **Q24. Use of Overriding? When should I override a method in real applications?**

- **Q25. If a method is marked as virtual, do we have to "override" it from the child class?**

Discover that in C#, a virtual method does not require overriding in derived classes; overriding is optional and, if not overridden, it still works, as shown by the Java example.

- **Q26. What is the difference between Method Overriding and Method Hiding?**

3. OOPS & C# - Abstract Class & Interface

Introduction

Explore the differences between abstract class and interface in .NET, review 11 questions, and practice scenario-based interview answers.

- **Q27. What is the difference between an Abstract class and an Interface? (V.IMP.)**

Compare abstract classes and interfaces in C#, detailing declaration versus definition, when interfaces can have method bodies, multiple inheritance, and constructors.

- **Q28. When to use Interface and when Abstract class in real applications?**

Explain when to use interfaces and abstract classes, using permanent and contractual employees as examples, highlighting interfaces as contracts and abstract classes for consistent implementations.

- **Q29. Why create Interfaces in real applications?**

- **Q30. Can we define the body of Interface methods?**

- **Q31. Can you create an instance of an Abstract class or an Interface?**
- **Q32. Can an Interface have a Constructor?**

Duration: 0:19

State that interfaces cannot have constructors and are used for inheritance, noting that interfaces cannot contain instance constructors.

- **Q33. Do abstract classes have Constructors in C#?**
- **Q34. What is the difference between abstraction and abstract class?**

Explain the difference between abstraction and abstract class, noting that abstraction shows only required details while hiding background ones, and that an abstract class is one way to implement abstraction.

- **Q35. Can an Abstract class be Sealed or Static in C#?**
- **Q36. Can you declare abstract methods as private in C#?**

In C#, you cannot declare abstract methods as private; abstract methods are only declarations and must be implemented by derived classes, otherwise a compiler error occurs.

- **Q37. Does an Abstract class support multiple Inheritance?**

Discover why an abstract class cannot support multiple inheritance in C#, and explore using multiple interfaces as an alternative for shared behavior and inheritance.

4. OOPS & C# - Access Specifiers, Boxing, Unboxing, String & StringBuilder

Introduction

Overview of access specifiers, type conversions, and string manipulation in C#.

- **Q38. What are Access Specifiers?**

- **Q39. What is an internal access modifier? Show example.**
- **Q40. What is the default access modifier in a class?**
- **Q41. What is Boxing and Unboxing? Where to use them in real applications?**

Explain boxing and unboxing in C# by converting value types to reference types and back, with an ArrayList illustrating boxing when adding integers and unboxing when retrieving.

- **Q42. Which one is explicit: Boxing or Unboxing?**
- **Q43. Is Boxing and Unboxing good for performance?**
Explore how boxing and unboxing impact performance, why integer-to-object and object-to-integer conversions slow down execution, and why using a generic list is preferable unless necessary.
- **Q44. What are the basic string operations in C#?**
- **Q45. What is the difference between "String" and "StringBuilder"?**
Compare strings and StringBuilder to understand immutability and mutability in C#. Demonstrate that strings are immutable and create new instances on modification, while StringBuilder mutates in place for efficient in-memory concatenation.
- **Q46. When to use String and when StringBuilder in real applications?**

Duration: 0:40

Use StringBuilder when you need to change a string multiple times to boost performance and avoid creating new strings; use string for infrequent updates, since it is lightweight.

- **Q47. What is String Interpolation in C#?**

5. OOPS & C# - Loops, Conditions, Exception Handling

Introduction

Covers control flow statements, conditions, and error-handling mechanisms in C#.

- **Q48. What are the Loop types in C#? When to use what in real applications?**

- **Q49. What is the difference between "continue" and "break" statements?**

Compare continue and break in loop control: continue transfers control to the loop start, skipping the rest of the iteration, while break exits the loop, showing 0, 1, 2 under continue.

- **Q50. What are the alternative ways of if-else conditions? When to use what?**

Explore alternative ways to express if-else logic by comparing switch statements and ternary operators, and learn when to use each for clean, readable, and concise code.

- **Q51. How to implement Exception Handling in C#?**

Learn to implement exception handling in C# using try, catch, and finally blocks to manage errors, log exceptions, and ensure final actions run regardless of errors.

- **Q52. Can we execute multiple Catch blocks?**

- **Q53. When to use Finally in real applications?**

Use the finally block to reliably dispose of resources, such as closing a SQL connection, ensuring the connection closes whether an error occurs and improving performance by avoiding garbage collection delay.

- **Q54. Can we have only a "Try" block without a "Catch" block?**

- **Q55. What is the difference between Finally and Finalize?**

- **Q56. What is the difference between "throw ex" and "throw"?**

Compare throw ex and throw to show how throw ex changes the stack trace, while throw preserves the full stack trace for accurate debugging and logs.

6. OOPS & C# - Generics & Collections

Introduction

Explore C# generics and collections through nine interview-style questions, covering array, list, hash table, dictionary, and differences questions, with a focus on enumerable and enumerator concepts, in about 18 minutes.

- **Q57. Explain Generics in C#? When and why to use them?**

Explore how generics in C# create type-independent, type-safe methods and classes using a type parameter T, while avoiding boxing and unboxing and boosting performance.

- **Q58. What are Collections in C# and what are their types?**

Explore collections in C#, their types and categories—normal, concurrent (thread-safe), and generic—along with examples like ArrayList, Hashtable, Dictionary, List, Queue, and Stack.

- **Q59. What is the difference between Array and ArrayList?**

Compare the difference between array and array list in .NET using code: arrays are strongly typed and fixed in size, while array lists store any type of item through boxing.

- **Q60. What is the difference between ArrayList and Hashtable?**

Compare array list and hash table by highlighting that array list stores values only, while hash table maps keys to values, enabling direct retrieval via keys.

- **Q61. What is the difference between List and Dictionary Collections?**

Duration: 1:38

Learn how lists differ from dictionaries by comparing array lists and generic lists, and see how dictionaries store key-value pairs as the generic hash table.

- **Q62. What is IEnumerable in C#?**

Explore how IEnumerable enables foreach iteration in C# by using a list of employees; see how List derives from IEnumerable to support enumeration.

- **Q63. What is the difference between IEnumerable and IEnumerator in C#?**

Compare enumerable and enumerator in C# by highlighting foreach-based iteration versus explicit GetEnumerator and MoveNext, and see how enumerable simplifies using the enumerator.

- **Q64. What is the difference between IEnumerable & IQueryable?**

Compare IEnumerable and IQueryable in C#, noting that IQueryable inherits from IEnumerable and enables server-side filtering for SQL queries, improving

performance by filtering at the database rather than loading data.

7. OOPS & C# - Constructors

Introduction

Explore constructors in C# through 12 concise questions, covering different constructor types in an easy-to-understand, eight-minute session.

- **Q65. What is a Constructor? When to use constructor in real applications?**

Understand what a constructor is and when to use it in applications. It runs when an object is created and sets default values; the constructor name must match the class name.

- **Q66. What are the types of constructors?**

Explore the types of constructors in C#—default, parameterized, copy, static, and private constructors, with step-by-step explanations.

- **Q67. What is a Default constructor?**

Learn how a default constructor in .NET is automatically created by the compiler when no other constructors exist, and why defining a parameterized constructor prevents this default from being generated.

- **Q68. What is a Parameterized constructor?**

Understand how a parameterized constructor accepts at least one parameter and prints the xparameter value, as shown with x equals 100.

- **Q69. What is a Static constructor? What is its use in real applications?**

Explore the static constructor in C# and its real-world use: it runs before any static member and before calling static methods via the class name, without object creation.

- **Q70. Can we have parameters or access modifiers in a static constructor?**

Discover why static constructors cannot have parameters or access modifiers in C#, as explained in the lecture.

- **Q71. What is a Copy constructor?**

A copy constructor is a constructor that creates an object by copying variables from another object of the same class, alongside the parameterized constructor in the class.

- **Q72. What is a Private constructor? What is its use?**

Learn what a private constructor does: it prevents inheritance and object creation,

suits classes with only static members, and enables the singleton pattern.

- **Q73. What is Constructor overloading?**

Define multiple constructors within a class with different parameter sets, enabling you to create objects with any number of parameters, just like method overloading.

- **Q74. What is a Destructor?**

- **Q75. Can you create an object of a class with a private constructor in C#?**

Explain why you cannot instantiate or inherit a class with a private constructor in C#, and how a private constructor prevents object creation.

- **Q76. If both base & child classes have constructors, which will be called first?**

Explain that when creating a derived object, the base class constructor fires first, then the derived constructor, clarifying the constructor call order in C#.

8. OOPS & C# - Method Parameters, Delegates & Events

Introduction

- **Duration:** 0:22

Explore questions on methods, parameters, delegates, and events, all tied to methods, with an especially important extension method question, in about 20 minutes.

- **Q77. What is a Method in C#?**

- **Q78. Difference between Pass by Value and Pass by Reference Parameters?**

Explain the difference between pass by value and pass by reference parameters using code examples, clarifying how the ref keyword alters variable addresses and outcomes.

- **Q79. How to return more than one value from a method in C#?**

Explore how to return multiple values from a method in C#, using ref and out parameters, or by returning a collection such as a list.

- **Q80. What is the difference between "out" and "ref" parameters?**

Explain the differences between out and ref parameters: out needs no prior initialization and must be assigned before returning, while ref requires initialization

and modifies an existing value.

- **Q81. What is the "params" keyword? When to use params keyword in real applications?**

Demonstrate how to use the params keyword in C# to pass a variable number of arguments, collected into an array and iterated with foreach for flexible method calls.

- **Q82. What are optional parameters in a method?**

Discover how optional parameters in a method work by using a single method with a default third parameter of 50; omit or pass the third parameter to set the value.

- **Q83. What are named parameters in a method?**

- **Q84. What are extension Methods? When to use them? (V. Imp.)**

Learn how extension methods add new functionality to existing classes without modifying source code. Explore static classes and this binding, with examples like a right substring extension.

- **Q85. What are Delegates in C#? When to use delegates in real applications?**

- **Q86. What are Multicast Delegates?**

Duration: 1:13

Learn how multicast delegates in C# hold references to more than one function and invoke them in sequence, enabling multiple methods like add and multiply to run.

- **Q87. What are Anonymous Delegates in C#?**

Discover how anonymous delegates in C# reference methods without names by providing the method body inline within the delegate declaration, i.e., an anonymous method.

- **Q88. What are the differences between Events and Delegates?**

Duration: 1:34

Learn the differences between events and delegates, with events acting as a notification mechanism built on delegates and securing multicast invocation like

button clicks.

9. OOPS & C# - Important Keywords

Introduction

Explore important C# keywords in this introduction, with 11 questions covering `as`, `readonly`, `const`, `enum`, `yield`, and more, in around 20 minutes.

- **Q89. What is the "this" keyword in C#? When to use it in a real application?**

The `this` keyword refers to the current instance of a class and distinguishes class fields from constructor parameters, preventing naming confusion between `this.id` and `id`.

- **Q90. What is the purpose of the "using" keyword in C#?**

Explain how the `using` statement guarantees disposal of resources like a database connection by automatically calling `Dispose` at the end of the block, for classes that implement `IDisposable`.

- **Q91. Can we use the `Using` keyword with other classes apart from `DB Connection`?**

- **Q92. What is the difference between "is" and "as" operators?**

Explain how the `is` operator checks an object's type and returns a boolean, while the `as` operator converts compatible reference types, not yielding a boolean.

- **Q93. What is the difference between "Readonly" and "Constant" variables?**

Compare `const` and `readonly` fields in C#, noting that `const` is a compile-time constant assigned at declaration and cannot change, while `readonly` can be set in declaration or constructor.

- **Q94. What is a "Static" class? When to use it?**

Explain the static class as a container for static members only, with no object creation or inheritance. Learn when to use it to access static methods via the class name.

- **Q95. What is the difference between "var" and "dynamic" in C#?**

- **Q96. What is the `Enum` keyword used for?**

Learn how the `enum` keyword groups constants into a single type, assigns automatic numeric values, and prevents inconsistencies when adding new

constants, with examples like levels and weekdays.

- **Q97. Is it possible to inherit Enum in C#?**

Explore why you cannot inherit an enum in C#. Enums are sealed class types, producing a compile-time error when inheritance is attempted, and coding-related concepts aid understanding.

- **Q98. What is the use of the Yield keyword in C#?**

Explore the yield keyword in C# and learn how yield return turns a get even numbers routine into a streaming iterator that generates values on the fly for efficiency.

10. OOPS & C# - LINQ

Introduction

Explore the final section of OOPS and C# with LINQ questions, focusing on basics commonly asked in interviews; complete these in about seven minutes.

- **Q99. What is LINQ? When to use LINQ in real applications?**

Learn what LINQ is: a query syntax in C# for retrieving data from in-memory collections. See how a query filters numbers 1 to 5 to values greater than two.

- **Q100. What are the advantages & disadvantages of LINQ?**

Explore the advantages and disadvantages of LINQ, including easier syntax, improved readability, and type safety, along with limited data-source support, maintainability, debugging complexity, and occasional performance gains.

- **Q101. What are Lambda Expressions? What is their use in real applications?**

- **Q102. What is the difference between First and FirstOrDefault methods in LINQ?**

11. .NET Framework - Basics

Introduction

Explore the basics of the .NET framework, including components, the GAC, reflection, serialization and deserialization, and globalization and localization, with a focused 15-minute overview.

- **Q103. What are the important components of the .NET framework?**

- **Q104. What is an Assembly? What are the different types of assembly in .NET?**

Explain how assemblies serve as the deployment unit, producing intermediate language inside a .dll in the bin folder, and distinguish private, public/shared, and satellite assemblies, which provide resources only.

- **Q105. What is GAC?**

Learn what GAC stands for (Global Assembly Cache) and how public assemblies are stored, with information on converting private assemblies to public using the GAC tool.

- **Q106. What is Reflection?**

Discover how reflection lets code access assembly metadata at runtime, including obtaining the current assembly version via `System.Reflection.Assembly.GetExecutingAssembly`, and using the `Type` class to inspect methods.

- **Q107. Serialization and Deserialization? What are the types of serialization?**

Learn how serialization converts objects to bytes for storage or transmission and how deserialization reconstructs objects from serialized data. Types include binary serialization, XML serialization, and JSON serialization.

- **Q108. What is meant by Globalization and Localization?**

Clarify how globalization designs software to support multiple cultures and languages, and how localization tailors a globalized app to a specific locale with an added language.

- **Q109. What are Windows Services?**

12. .NET Framework - Garbage Collection

Introduction

Explore garbage collection in .NET after mastering .NET framework basics, with five quick questions. Learn what happens in the background; no coding required, about nine minutes total.

- **Q110. What is Garbage Collection (GC)?**

- **Q111. What are Generations in garbage collection?**

- **Q112. What is the difference between "Dispose" and "Finalize"?**

- **Q113. What is the difference between "Finalize" and "Finally" methods?**

Differentiate the Finalize method from the finally block in .NET/C#. Finalize is used by the garbage collector for cleanup; finally runs code regardless of exceptions.

- **Q114. Can we force Garbage Collector to run?**

Explore whether you can force the .NET garbage collector to run using GC.Collect, why it's discouraged due to performance costs, and why using the Dispose method is preferred.

13. .NET Framework - Threading

Introduction

Explore core threading concepts in .NET, including multi-threading, synchronous and asynchronous programming, and the task, async, and await pattern, through a concise set of five interview questions.

- **Q115. What is the difference between Process and Thread?**

Explain the difference between a process and a thread: a process has its own memory space and resources, while a thread shares memory with other threads in the same process.

- **Q116. Explain Multithreading?**

Explain multithreading by comparing single and multi-threaded processes and show how to run multiple threads, using C# and System.Threading for tasks like downloading data and updating a progress bar.

- **Q117. What is the difference between synchronous and asynchronous programming?**

Explore the difference between synchronous and asynchronous programming in .NET/C#, where synchronous code executes methods sequentially on a single thread, while asynchronous code uses Task.Run to run them in parallel.

- **Q118. Difference between Threads & Tasks? Advantages of Tasks over threads?**

Explore the difference between threads and tasks in .NET, and learn why tasks simplify multithreading by wrapping threads, enabling easier exception handling, returning values, and task chaining.

- **Q119. What is the role of Async and Await?**

Explore how async and await enable asynchronous programming in C#, contrasting it with synchronous execution and showing how dependent methods coordinate with tasks and threads.

14. SQL - Basics

Introduction

The lecture introduces SQL as a new .NET interview topic, covering eight basic questions on DBMS and RDBMS constraints, including primary key, unique key, triggers, views, having, and where.

- **Q120. What is the difference between DBMS and RDBMS?**

Explain how DBMS stores data as files with no relationships, while RDBMS uses tabular data with relationships and normalization, exemplified by XML vs MySQL and SQL Server.

- **Q121. What is a Constraint in SQL? What are its types?**

- **Q122. What is the difference between Primary key and Unique key?**

A primary key cannot be null and creates a clustered index. A unique key allows one null value and uses a non-clustered index; a table has one primary key and multiple unique keys.

- **Q123. What are Triggers and types of triggers?**

Understand triggers as stored programs that fire on insert, update, or delete to track changes, using DML triggers like AFTER and INSTEAD OF.

- **Q124. What is a View?**

Describe how a view acts as a virtual table defined by a query, selecting a subset of data from one or more tables, and leverage indexing and security benefits.

- **Q125. What is the difference between Having clause and Where clause?**

Compare WHERE and HAVING clauses, noting that WHERE appears before GROUP BY and HAVING follows it; WHERE cannot use aggregate functions, while HAVING can; with GROUP BY, we prefer HAVING.

- **Q126. What is a Sub query or Nested query or Inner query in SQL?**
- **Q127. What is an Auto Increment / Identity column in SQL Server?**

Understand auto increment and identity columns in SQL Server, which automatically generate unique values for new records, usually for primary keys, starting at 1 and incrementing by 1.

15. SQL - Joins & Indexes

Introduction

Explore eight SQL questions on joins and indexes, including types of join, self join, and clustered versus non-clustered indexes. The section runs about 13 minutes.

- **Q128. What are Joins in SQL?**

Explore how a join clause combines rows from two or more tables based on a related column, as shown with tables A and B producing student id, name, and department.

- **Q129. What are the types of Joins in SQL Server?**

Learn the types of joins in SQL Server, including left, right, full outer joins, and inner join, with guidance on when and how to use each to match rows.

- **Q130. What is a Self-Join?**

- **Q131. What are Indexes in SQL Server?**

- **Q132. What is a Clustered index?**

Understand how a clustered index defines the physical order of table records, why a table can have only one clustered index, and how a primary key creates one automatically.

- **Q133. What is a Non-Clustered index?**

Define a non-clustered index as a separate structure from table data, unlike clustered indexes stored with data. A table can have multiple non-clustered indexes, like end-of-book indexes mapping values to page numbers.

- **Q134. What is the difference between Clustered and Non-Clustered index?**

Explain the difference between clustered and non-clustered indexes: clustered

indexes determine the order and are stored with the data, while non-clustered indexes are stored separately and can be many.

- **Q135. How to create Clustered and Non-Clustered index in a table?**

Learn to create clustered and non-clustered indexes on a table with syntax, including index name, table, and column lists; note how a primary key often creates a clustered index automatically.

- **Q136. In which column will you apply indexing to optimize this query?**

Learn how to optimize queries by creating non-clustered indexes on WHERE condition columns, such as name, and avoid indexing select columns or before the LIKE to speed up performance.

16. SQL - Stored Procedures, Functions & Others

Introduction

Explore PL/SQL concepts, focusing on stored procedures and functions, differences between them, and CTEs, to cover almost 100% of SQL interview questions in about 19 minutes.

- **Q137. What is the difference between Stored Procedures and Functions?**

Differentiate stored procedures and functions: procedures may or may not return a value and can modify data; functions always return a value, take only input parameters, and cannot call procedures.

- **Q138. How to optimize a Stored Procedure or SQL Query?**

- **Q139. What is a Cursor? Why avoid them?**

Discover what a cursor is, how it traverses rows via DECLARE, OPEN, FETCH, CLOSE, and DEALLOCATE steps, and learn why cursors hurt performance and should be avoided.

- **Q140. What is the difference between SCOPE_IDENTITY and @@IDENTITY?**

Compare SCOPE_IDENTITY and @@IDENTITY: both capture the last identity value, but @@IDENTITY can differ within a session; SCOPE_IDENTITY returns the last identity in the same scope, ensuring reliability for stored procedures.

- **Q141. What is CTE in SQL Server?**

- **Q142. What is the difference between Delete, Truncate, and Drop commands?**

- **Q143. How to get the Nth highest salary of an employee?**

Explain how to get the nth highest salary by selecting the top three salaries in descending order, reversing the results to ascending, and picking the top result.

- **Q144. What are ACID properties?**

Atomicity ensures a transaction's statements all execute or none do. Consistency enforces valid data types and constraints; isolation keeps transactions from affecting each other; durability guarantees committed data survives failures.

- **Q145. What are Magic Tables in SQL Server?**

Explain how magic tables in SQL Server automatically track inserted, updated, and deleted rows as temporary background tables (inserted and deleted), and their use with triggers from a DBA perspective.

17. ADO.NET and Entity Framework

Introduction

Concepts related to database connectivity, architectural patterns, and ORMs in .NET.

- **Q171. What are the main components of ADO.NET?**

Identify ADO.NET components—dataset, data adapter, and data reader—and describe how a dataset stores data, how a data adapter fills it, and how a data reader reads results.

- **Q172. What is Connected architecture and Disconnected architecture?**

Compare connected and disconnected architecture, using a data adapter to fetch, manipulate, and resubmit data offline, and note the disconnected approach as faster and more robust than direct database access.

- **Q173. What are the different Execute Methods of ADO.NET?**

Explore the three execute methods in ADO.NET: ExecuteScalar returns a single value, ExecuteNonQuery handles insert, update, and delete, and ExecuteReader retrieves data only.

- **Q174. What are the Authentication techniques used to connect to SQL Server?**

- **Q175. What is ORM? What are the different types of ORM?**

Define ORM as object-relational mapping that maps application objects to database

tables, and note Entity Framework, NHibernate, and Dapper as .NET ORM types, with Entity Framework most used.

- **Q176. What is Entity Framework?**

Explore how the open-source object-relational mapping tool from Microsoft, Entity Framework, wraps .NET and maps domain objects to database tables, automating relational data handling and reducing coding effort.

- **Q177. How will you differentiate ADO.NET from Entity Framework?**

Duration: 2:32

Entity Framework shortens code and auto-generates SQL from LINQ, offering database independence; ADO.NET relies on manual SQL and is tightly coupled to SQL Server, with EF typically slower.

- **Q178. How does Entity Framework work? OR How to setup EF?**

Duration: 2:04

Install the Entity Framework package, create a student model and a SchoolContext with a DbSet for students, inject the context, build a students list, add them, and call SaveChanges.

- **Q179. What is meant by DbContext and DbSet?**

Duration: 1:15

Learn how a DbContext enables communication between the database and entity classes, and how DbSets represent entity sets used for create, read, update, and delete operations.

- **Q180. What are the different types of development approaches used with EF?**

Duration: 2:12

- **Q181. What is the difference between LINQ to SQL and Entity Framework?**

18. Web API - Basics

Introduction

Foundations of building and consuming lightweight HTTP services in .NET.

- **Q182. What is Web API? What is the purpose of Web API?**

- **Q183. What are Web API advantages over WCF and web services?**

Compare Web APIs' advantages over WCF and web services by highlighting ease of use, speed, lightweight HTTP and JSON, and cross-platform hosting.

- **Q184. What are HTTP verbs or HTTP methods?**

Explore how HTTP verbs drive GET, POST, PUT, and DELETE to fetch, create, update, and remove resources from a web API and exchange JSON payloads.

- **Q185. What is the difference between REST API and Web API?**

REST is not another technology; it's a set of guidelines for inter-application communication. When a Web API follows these guidelines, it can be called a REST API (Representational State Transfer).

- **Q186. What are REST guidelines? What is the difference between Rest and Restful?**

- **Q187. Is it possible to use WCF as Restful services?**

Develop RESTful services with WCF by following REST guidelines in a WCF project to implement RESTful capabilities.

- **Q188. How to consume Web API from a .NET MVC application?**

- **Q189. What is the difference between Web API and MVC Controller?**

Explain the difference between Web API and MVC controllers: Web API controllers derive from ApiController and lack view support; MVC controllers derive from Controller and provide view support.

19. Web API - Authentication & JWT

Introduction

Explore JWT authentication as the centerpiece of web API authentication, covering six questions on authentication techniques and expanding your knowledge of web API security in about nine minutes.

- **Q190. What are the types of authentication techniques in web API?**

Explore six authentication techniques for web APIs, including token-based authentication, basic, API key, certificate, Windows, and multi-factor methods, with an emphasis on token-based approaches.

- **Q191. What is Basic Authentication in Web API?**

Learn how basic authentication works in web API by sending user credentials in a POST request for verification, and understand why this method is insecure due to plain text transmission.

- **Q192. What is API Key Authentication in Web API?**

- **Q193. What is Token-based authentication?**

- **Q194. What is JWT Authentication?**

Explain how JWT authentication uses token-based security by issuing a JSON Web Token after validating user credentials, transmitting the token in the authorization header, and validating its signature.

- **Q195. What are the parts of a JWT token?**

Describe the three parts of a JWT token—header, payload, and signature—and how the header defines type and algorithm, the payload holds issuer, subject, exp, iat, and the signature verifies integrity.

- **Q196. Where does the JWT token reside in the request?**

Explain that the JWT token resides in the request header, specifically the Authorization header, where the key is Authorization and the token serves as the value.

20. Web API - More

Introduction

Explore seven web API questions focusing on action return types such as HttpResponseMessage and IActionResult, content negotiation, media type formatter, and response codes in about 12 minutes.

- **Q197. How to test Web API? What are the tools?**

Test web API inputs and outputs using Postman, Swagger, and Fiddler. Recognize that web APIs lack a UI, so these tools provide practical testing options, as shown

by Postman.

- **Q198. What are the main Return Types supported in Web API?**

Explain the main return types in web API controllers, highlighting HttpResponseMessage and IActionResult as primary wrappers, plus JSON, XML, file stream, empty, status code, and redirect results.

- **Q199. What is the difference between HttpResponseMessage and IHttpActionResult?**

Compare HttpResponseMessage and IHttpActionResult in Web API, showing v1.0 uses HttpResponseMessage and manual JSON content, while v2.0 uses IHttpActionResult for NotFound and Ok responses with automatic JSON serialization.

- **Q200. What is the difference between IActionResult and IHttpActionResult?**

Clarify the difference between IActionResult and IHttpActionResult: IHttpActionResult is used in ASP.NET Web API, while IActionResult is used in ASP.NET Core Web API.

- **Q201. What is Content Negotiation in Web API?**

- **Q202. What is MediaTypeFormatter class in Web API?**

Explain how the media type formatter in ASP.NET Web API converts objects to JSON or XML for responses, and how to create custom formatters that extend the base formatter.

- **Q203. What are Response Codes in Web API?**

21. .NET Core - Basics

Introduction

Explore nine basic .NET Core questions and establish core concepts, then preview dependency injection in upcoming sections, all within about 17 minutes.

- **Q204. What is .NET Core?**

.NET Core introduces a completely new framework distinct from the .NET Framework and Mono for Xamarin; it is free, open source, and developed by Microsoft.

- **Q205. What is .NET Standard?**

.NET Standard is a set of rules common across all .NET frameworks, not a framework itself. A framework becomes .NET Standard compliant when it follows these rules and APIs.

- **Q206. What are the advantages of .NET Core over .NET framework?**

Explore the advantages of .NET Core over .NET Framework, including cross-platform, open source, free for companies, flexible hosting, built-in dependency injection, multiple IDEs, and faster performance.

- **Q207. What is the role of Program.cs file in ASP.NET Core?**
- **Q208. What is the role of ConfigureServices method?**
- **Q209. What is the role of Configure method?**
- **Q210. Describe the complete Request Processing Pipeline for ASP.NET Core MVC?**

Trace the complete request processing pipeline for ASP.NET Core MVC, from middleware and routing to the controller, action, and view rendering. Web APIs omit the view and return JSON strings.
- **Q211. What is the difference between .NET Core and .NET 5?**

Explain how .NET 5 represents the future of .NET Core, with 'Core' dropped from the name, and note that .NET 6 is the latest version of .NET Core.
- **Q212. What is Metapackage? What is the name of Metapackage in ASP.NET Core?**

22. .NET Core - Dependency Injection

Introduction

Covers inversion of control, dependency injection patterns, and decoupling components in ASP.NET Core.

- **Q213. What is Dependency Injection?**

- **Q214. How to implement Dependency Injection in .NET Core?**

Explore dependency injection in .NET Core to replace tight coupling with an IStudent interface, constructor injection, and service registration for swappable math and science classes.

- **Q215. What are the advantages of Dependency Injection in .NET Core?**

Discover how dependency injection in .NET Core enables loose coupling, flexible behavior changes, easier unit testing with interfaces, independent modules, and reusable components.

- **Q216. How to use Dependency Injection in Views in ASP.NET Core?**

23. .NET Core - Service Lifetimes, Middleware & Hosting

Introduction

Focuses on internal execution, request delegates, server models, and lifetime scopes.

- **Q217. What are the types of Service Lifetimes of an object/instance in ASP.NET Core?**
- **Q218. What is AddSingleton, AddScoped and AddTransient method?**
- **Q219. What is Middleware in ASP.NET Core? What is custom middleware?**
- **Q220. How is ASP.NET Core Middleware different from HttpModule?**
- **Q221. What is a Request Delegate?**
- **Q222. What is Run(), Use() and Map() method?**
- **Q223. What are the types of Hosting in ASP.NET Core? What is In-process and Out-of-process?**

- **Q224. What is Kestrel? What is the difference between Kestrel and IIS?**

24. .NET Core - Routing, Files, CORS & More

Introduction

Exploration of app configuration, static assets, cross-origin security, routing, and caching.

- **Q225. What is Routing? Explain attribute routing in ASP.NET Core?**
- **Q226. Explain default project structure in ASP.NET Core application?**

- **Q227. How does ASP.NET Core serve static files?**

Enable static file serving in ASP.NET Core by configuring the UseStaticFiles middleware in startup to serve files from the wwwroot folder before the MVC controller and authorization.

- **Q228. What are the roles of Appsettings.json and Launchsettings.json files?**

Store configuration with appsettings.json, including database connection strings and global variables, for the running app. Use launchSettings.json to define startup parameters such as the application URL, port, and hosting.

- **Q229. What are the techniques to save configuration settings in .NET Core?**

Explore techniques to save configuration settings in ASP.NET Core, including appsettings.json and Azure Key Vault, along with environment variables, in-memory .NET objects, command-line arguments, and custom providers.

- **Q230. What is CORS? Why is CORS restriction required? How to fix CORS errors?**

Understand what CORS is (Cross-Origin Resource Sharing), why browsers restrict cross-domain requests for security, and how to fix CORS errors in .NET Core by enabling CORS in startup and services.

- **Q231. What is In-Memory caching & Distributed Caching? When to use what?**

Compare in-memory caching and distributed caching: in-memory stores data on the same server; distributed caching uses a separate Redis cache. Use IMemoryCache for small apps and Redis for microservices.

- **Q232. How to handle errors in ASP.NET Core?**

Explore environment-based error handling in ASP.NET Core, switching from detailed exception pages in development to a custom error page in staging and production, configured via startup and launch settings.

- **Q233. What are Razor pages in .NET Core?**

25. SOLID Principles

Introduction

Explore SOLID principles in depth, covering SRP, OCP, LSP, ISP, and DIP, plus DRY, through seven interview questions for experienced .NET developers.

- **Q234. What are SOLID Principles? How are they different from Design Patterns?**

Explains SOLID principles as essential, abstract guidelines for flexible, maintainable software, contrasts them with design patterns as concrete solutions, and shows when to apply patterns like factory or singleton.

- **Q235. What is the Single Responsibility Principle (SRP)?**

Explain the single responsibility principle: a class should have one responsibility and one reason to change. The lecture uses the employee and repository examples to illustrate SRP and its benefits.

- **Q236. What is the Open-Closed Principle (OCP)?**

Explore the open-closed principle: extend account behavior via interfaces rather than modifying classes, and use SRP with separate saving and current account implementations for easy extension and testing.

- **Q237. What is the Liskov Substitution Principle (LSP)?**

Demonstrate the Liskov substitution principle by showing how derived classes override base methods and must replace the base class without breaking the application, keeping base methods applicable.

- **Q238. What is the Interface Segregation Principle (ISP)?**

- **Q239. What is the Dependency Inversion Principle (DIP)?**

- **Q240. What is the DRY principle?**

Learn the DRY principle: "Don't Repeat Yourself". Write code once and reuse it via methods and inheritance, using a base class so derived classes share the method.

26. Design Patterns

Introduction

Explore design patterns for experienced candidates with ten questions, covering the basics and focusing on the singleton and factory method patterns (3–4 patterns) in about 25 minutes.

- **Q241. What are Design Patterns and what problem do they solve?**

Identify design patterns as reusable solutions for common software design problems and distinguish them from generic solutions by how classes communicate in a new application.

- **Q242. What are the types of Design Patterns?**

Identify the three design pattern categories: creational, structural, and behavioral. Explore popular patterns like factory method, builder, abstract factory, singleton, adapter, decorator, proxy, observer, command, state, and visitor.

- **Q243. What are Creational Design Patterns?**

Explore creational design patterns that optimize object creation, such as factory method, builder, abstract factory, and singleton, to avoid widespread changes and testing issues in large applications.

- **Q244. What are Structural Design Patterns?**

Understand structural design patterns that manage the structure of classes and interfaces, using the adapter pattern as a key example and noting decorator, facade, and proxy patterns.

- **Q245. What are Behavioral Design Patterns?**

- **Q246. What is the Singleton Design Pattern?**

Ensure one instance with the singleton design pattern, a pattern shared by multiple requests, implemented through a sealed class, a static instance, a private constructor, and a GetInstance method.

- **Q247. How to make the singleton pattern thread-safe?**

- **Q248. What is the Factory pattern? Why use the factory pattern?**
- **Q249. How to implement the Factory Method pattern?**
- **Q250. What is the Abstract Factory pattern?**

Explore the abstract factory pattern, a creational design pattern that provides interfaces for creating related families of objects without specifying their concrete classes, effectively a factory of factories.

27. Array Coding Problems

Array Coding Problems

Overview of problem-solving techniques for arrays.

- **Q251. What are the must-know prerequisites for solving basic coding problems?**

Identify must-know prerequisites for solving basic coding problems, including data types and variables, classes and methods (main method and functions), operators, control statements, and arrays and strings.

- **Q252. What is the common approach for solving coding problems?**

Apply a universal approach to coding problems: understand the problem, set input and output, implement in a function, test edge cases, and add comments for readability.

- **Q253. Write a function to calculate the sum of all elements in an array.**

Define a function to compute the sum of all array elements by iterating with a for loop, initializing the sum to zero, and returning the result.

- **Q254. Write a function to calculate the average of an array of numbers.**

Write a function to calculate the average of an array by summing elements and dividing by the array length, then learn step-by-step from simple to hard problems.

- **Q255. Write a function to find the smallest number in an array.**

Write a function to find the smallest number in an array by assuming the first element as the smallest, then iterating with a for loop and returning the smallest value.

- **Q256. Write a function to find the largest number in an array.**

Write a function to find the largest number in an array by initializing with the first element and updating when a larger value appears—a key interview tip.

- **Q257. Write a function to find the second largest number in an array.**

Duration: 6:30

Learn to implement a function that finds the second largest number in an array by iterating and updating the largest and second largest values, including simple and edge cases.

28. Array Coding Problems Using Functions

Array Coding Problems Using Functions

Implementation-focused array interview challenges.

- **Q258. What are the top 5 important array functions used in coding problems?**

Discover the top five important array functions used in coding problems: `string.Join`, `Sort`, `Reverse`, `Fill`, and `Copy`, with practical examples.

- **Q259. Write a function to check whether two arrays are the same or not.**

Use the `SequenceEqual` method to compare two arrays and return `true` if identical, or implement a simple `for` loop to check elements.

- **Q260. Function to check if a given array is sorted in ascending order?**

Write a function to check if a given array is sorted in ascending order, using a `for` loop to compare consecutive elements and return `true` or `false` early when unsorted.

- **Q261. Write a function to merge two arrays into a single sorted array.**

Learn to merge two arrays into a sorted array by creating a merged array, copying elements from both inputs, and then sorting, with an alternative `for`-loop method.

- **Q262. Write a function to remove a specific element from an array.**

Write a function to remove a specific element from an array by counting occurrences, creating a smaller array, and filling it with non-target elements, then print with `string.Join`.

29. String Coding Problems

String Coding Problems

Tackle four string manipulation coding problems with a focus on problem-solving approach, building interview-ready strategies for string challenges.

- **Q263. Write a function that counts the number of characters in a string?**

- **Q264. How to iterate a string?**

- **Q265. Write a function that returns the reverse of a string?**

Learn to reverse a string efficiently by iterating from the end and appending characters with a `StringBuilder`, highlighting performance benefits and edge-case considerations.

- **Q266. Write a function that checks whether a given string is a palindrome?**

Write a function to check if a string is a palindrome by reversing it and comparing it with the original. Use the `Equals` method to determine equality and return a `boolean`.

30. String Coding Problems Using Functions

String Coding Problems Using Functions

Explore string coding problems using built-in string functions in C# and .NET, solving challenges by applying those functions, pausing to ponder, writing code, and validating results.

- **Q267. What are the important methods of the `String` class?**

- **Q268. Write a function that returns the longest word in the sentence.**

- **Q269. Write a function to remove all whitespace characters from a string.**

Write a function to remove all whitespace from a string by replacing whitespace with an empty string using the `string.Replace` method; emphasize meaningful names for variables and functions.

- **Q270. Write a function that counts the number of vowels in a string?**

Write a function to count vowels in a string by iterating characters, checking membership in a vowel set (lowercase and uppercase), and returning the count.

- **Q271. Write a function that checks whether two strings are anagrams or not?**

Check if two strings are anagrams by converting them to character arrays, sorting, and comparing the results. Implement a function that returns true when both sorted arrays match.

31. Number Coding Problems

Number Coding Problems

Algorithmic logic and basic mathematical implementations in C#.

- **Q272. Write a function to calculate the factorial of a number.**

Duration: 2:34

Learn to implement a factorial function in code, initializing the result to 1, looping from 1 to n , multiplying, and returning the final product, with the zero factorial case handled.

- **Q273. What is the difference between $++i$ and $i++$?**

Explore the difference between pre-increment ($++i$) and post-increment ($i++$), with explanations of immediate versus after-statement effects on results.

- **Q274. Write a function that checks whether a number is prime or not?**

Write a function to check if a number is prime with a loop from 2 to $n/2$, returning true when no divisor is found and false for $n \leq 1$.

- **Q275. How to swap two numbers in C#?**

Demonstrate swapping two numbers in C# using a temp variable, with inline code that assigns a to temp, then b to a , then temp to b , for interview prep.

- **Q276. Write a function to calculate the GCD for two numbers.**

Implement a GCD function by looping from the smaller number down to 1 and returning the first divisor that divides both, as 30 and 18 yield 6.

- **Q277. Write a function to sum the digits of a number.**

Write a function to sum the digits of a number using modulo ten and repeated division by ten with a while loop, returning the total when the number reaches zero.

- **Q278. Write a function to calculate the Fibonacci sequence up to a given number.**

Write a function to calculate the Fibonacci sequence up to a given number by

iterating with two initial values zero and one, updating a and b , and printing the sequence.